Reviewer Pack — Minimal Rank of Monoidal Categories vs Resolution Proof Leng…

Entry b7447e20d3f6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 19:56:46 UTC

Minimal Rank of Monoidal Categories vs Resolution Proof Length for Tseitin Formulas

Entry ID: `b7447e20d3f6` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 19:56:46 UTC

1. Conjecture

Field A (mathematical branch): Monoidal Category Theory **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity (for Tseitin Formulas)

Statement:

> ['For a given Tseitin formula F with n variables and m clauses, the minimal rank of any monoidal category C that is equivalent to the morphism complexity category of F is at least as large as the length of the shortest resolution proof for F.', 'Formally: ∀F ∈ Tseitin(n, m), ∃C ⊆ MonCat such that C ≡ MC(F), and rank(C) ≥ length(ResolutionProof(F)).']

Rationale (proposer's reasoning):

> ['Monoidal categories are a rich source of algebraic structures that can encode computational processes. Since resolution proof complexity is a measure of the difficulty of proving formulas, the conjecture posits a direct relationship between these algebraic structures and the inherent complexity of their associated Tseitin formulas.', 'This bridge could expose new structural insights into the nature of computation by demonstrating how algebraic properties of monoidal categories can be leveraged to reason about proof lengths.']

Taxonomy category: `Monoidal Categories × Resolution Proof Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `362fad06bfcfe4b3`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all generated Tseitin formulas F with n variables and m clauses (n ≤ 40), the minimal rank of the monoidal category C equivalent to MC(F) meets or exceeds the length of the shortest resolution proof for F by at least a threshold of 2 standard deviations above the mean. The criterion is falsified if any seed produces a case where rank(C) < length(ResolutionProof(F)) or the difference is less than 2 standard deviations below the mean.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Minimal Rank Monoidal Category Resolution Proof Complexity Tseitin Formula` - `Monoidal Category Theory Resolution Proof Length Tseitin Formulas` - `Tseitin Formula Morphism Complexity Category Minimal Rank`

Top relevant hits considered: - [http://arxiv.org/abs/2201.07527v2] The free compact closure of a symmetric monoidal category - [http://arxiv.org/abs/1712.08276v2] Braided skew monoidal categories - [http://arxiv.org/abs/2503.17274v3] Composable Uncertainty in Symmetric Monoidal Categories for Design Problems (Extended Version) - [http://arxiv.org/abs/2308.00286v3] Morphisms from projective spaces to flags of minimal parabolic subgroups - [http://arxiv.org/abs/1308.1443v1] Category of asynchronous systems and polygonal morphisms - [http://arxiv.org/abs/1310.7050v3] Rank-finiteness for modular categories

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def parse_tseitin(formula):
        n = 0
        clauses = []
        for char in formula:
            if char.startswith('v'):
                n = max(n, int(char[2:]))
            elif char.startswith('~'):
                continue
            else:
                clauses.append(char)
        return n, clauses

    def resolution_proof_length(formula):
        n, clauses = parse_tseitin(formula)
        proof_length = len(clauses)  # Simplified for demonstration purposes
```

```
        return proof_length

    def morphism_complexity_category(n, m):
        # Placeholder implementation; replace with actual computation
        category_rank = n + m  # Example rank calculation
        return category_rank

    formula = ''.join(random.choices(['v', '~'], k=10))  # Simplified for demonstration purposes
    proof_length = resolution_proof_length(formula)
    category_rank = morphism_complexity_category(5, 3)  # Simplified for demonstration purposes

    metric_name = "minimal_rank"
    metric_value = category_rank
    instances_tested = 1
    conjecture_holds = category_rank >= proof_length
    counterexample = "" if conjecture_holds else f"Formula: {formula}, Category Rank: {category_ra

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
        std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for res
        support_fraction = 1.0
    else:
        mean_metric_value = sum(result["metric_value"] for result in results if result["conjecture
        std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for res
        support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(result

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='<not applicable>' first_failing_seed={first_fail
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f5ea0eda.py", line 67, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f5ea0eda.py", line 44, in run_trial
    proof_length = resolution_proof_length(formula)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f5ea0eda.py", line 34, in resolution_pr
    n, clauses = parse_tseitin(formula)
                 ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f5ea0eda.py", line 26, in parse_tseitin
    n = max(n, int(char[2:]))
              ^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: ''
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which prevents us from verifying the conjecture's support or falsification. | next: Investigate and fix the crash in the test code to continue testing the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13121
2	propose	ollama_remote	glm4:latest	0	0	11310
3	preregistration	ollama_remote	glm4:latest	0	0	6240
4	novelty	ollama_remote	glm4:latest	0	0	4587
5	novelty	ollama_remote	glm4:latest	0	0	9178
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19762
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10362
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11851
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9458
10	judge	ollama_remote	glm4:latest	0	0	10561

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 106430 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/b7447e20d3f6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b7447e20d3f6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b7447e20d3f6.tar.gz` (if generated)